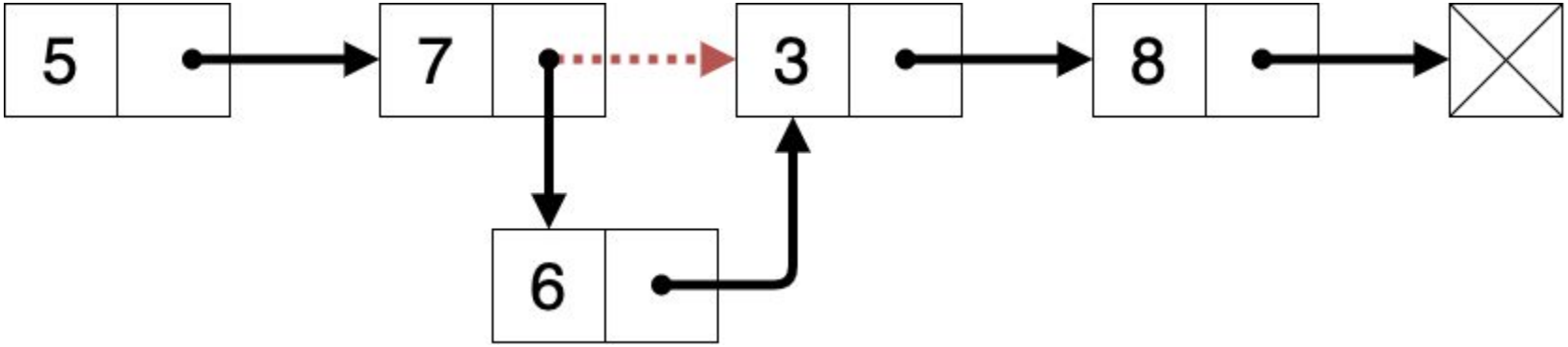
Delete and Insert After a Value in a Linked List

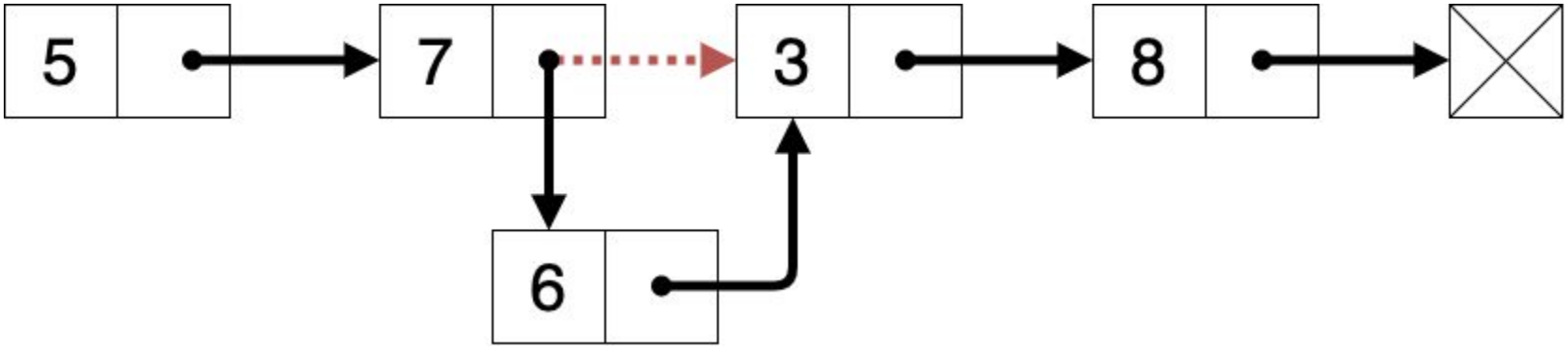
Kevin Browne

Remember one of our original motivations for using a linked list was the ease of inserting nodes into the middle of the list!



Inserting after a value

- Let's make a function to insert a node with a given value **after** the first node in the linked list with another given value
  - i.e. insert a node with value '6' after the first node with value '7'



Inserting after a value

- If the list is empty, we insert a node with the given value
- If the list does not contain a node with the specified value to insert the node after, we will insert the new node at the tail
- We don’t need to make the function this way, these are just default behaviours that we’re deciding!

Insert After Function

```
Node *insert_after(Node *head, int new_value,
                   int after_value)
{
  Node *new_node = calloc(1, sizeof(Node));
  new_node->value = new_value;
  if (head == NULL) return new_node;
  else
  {
    Node *current = head;
    while (current->next != NULL)
    {
      if (current->value == after_value)
      {
        new_node->next = current->next;
        current->next = new_node;
        return head;
      }
      else current = current->next;
    }
    new_node->next = current->next;
    current->next = new_node;
    return head;
  }
}
```

Test code

```
Node *list = NULL;

list = insert_at_tail(list, 5);
list = insert_at_tail(list, 9);
list = insert_at_tail(list, 1);
list = insert_at_tail(list, 8);
list = insert_at_tail(list, 4);

printf("List before insert:\n");
print_list(list);
insert_after(list, 20, 8);
printf("List after insert:\n");
print_list(list);
```

Test results

```
List before insert:
Node 0: 5
Node 1: 9
Node 2: 1
Node 3: 8
Node 4: 4
List after insert:
Node 0: 5
Node 1: 9
Node 2: 1
Node 3: 8
Node 4: 20
Node 5: 4
```

Delete a list

- How can we delete an entire list?
  - For example, because we are done working with a list and no longer need it at at all!
- We can recursively traverse the list, free-ing each node as we traverse the list
  - Until we reach the tail (or the list is empty), at which point we are don
- For consistency's sake with other functions, we should have the function return NULL
  - i.e. the "new list", which is now empty

Delete list function

```
Node *delete_list(Node *node)
{
  if (node != NULL)
  {
    delete_list(node->next);
    free(node);
  }
  return NULL;
}
```

Test code

```
list = delete_list(list);
printf("List after delete:\n");
print_list(list);
```

Test results

```
List after insert:
Node 0: 5
Node 1: 9
Node 2: 1
Node 3: 8
Node 4: 20
Node 5: 4
List after delete:
bash-3.2$
```